### 1. 架构

我们要做的架构是一个**本地运行的 B/S 混合架构**。 简单来说，就是**把 UE4 当作后端的一个渲染器来用**。 整个系统的核心分为三块：

- **最前端（Web网页）**：
  - 这是给用户操作的界面,类似**云电脑/云游戏**。用户在浏览器里输入参数（比如哪里起火、有多少人），同时这个网页会像看直播一样，实时显示 UE4 渲染出来的 3D 画面。并且还有像游戏一样控制视角自由移动等功能，都依靠于实时画面传输。
  - 也就是说这个**真**前端只用网页即可，渲染的是2维画面，不承担3维渲染职责。
- **中台（UE4 客户端）**：它在后台（架构上可属后端，但我们演示结果时不需要分离机器所以实际中台和真后端都在同一台电脑上）运行，虽然有**但**不直接给用户看窗口。它负责接收仿真数据，把地铁站、水、火渲染出来，然后通过 **Pixel Streaming（像素流送（maybe））** 技术把画面推给网页。
- **后端（Node.js 仿真服务）**：这是仿真的大脑。它负责跑算法（算水怎么流、人往哪跑），并对接大模型 API 获取决策建议。

### 2. 为什么要这么设计？

> 如果是纯 `UE4` 客户端，`UI` 和逻辑耦合太死，老师那边过不了关，以后没法转型；如果是用 `Three.js` 在网页重写渲染，那我们要花大量时间去手写渲染器，做不出 `UE4` 那种高质量的水体和烟雾效果。

**这套方案：**

- **拓展性**：我们的控制逻辑全在 Web 和 Node.js 里，UE4 只负责画画。以后如果想把 UE4 换成专业的工业仿真软件，网页和后端代码基本不用动，符合老师要求的“架构解耦”。
- **画质**：我们可以继续发挥 UE4 在光影、材质上的强项，不用去啃 WebGL 的生肉。
- **规避网络延迟**：虽然这套叫“云渲染”架构，但我们演示时是在**一台高性能电脑上**同时跑这三个程序（Localhost），不存在网络卡顿的问题，操作体验和本地游戏一样流畅。

### 3. 技术可行性分析（大模型结论）

这个方案在技术栈上是完全打通的：

- **前端**：Vue 或普通 HTML 都可以，主要集成 UE 官方提供的 `webRtcPlayer.js` 就能显示画面，UI 开发难度低。
- **后端**：我们已经跑通了 Node.js 和 UE4 的 HTTP 通信（倒水桶 Demo），接下来只需要增加数据量和频率。
- **渲染端**：UE 4.26 自带 Pixel Streaming 插件，这是一个成熟的工业标准功能，不需要我们自己造轮子。
- **大模型**：本质上就是后端多发一个 HTTP 请求给 GPT/文心一言，拿回文本指令后再解析成仿真参数即可。

### 4. 分工安排

基于这个架构，关键时都要有明确的产出接口：

- **A：仿真算法与真后端**
  - **核心任务**：写 Node.js 服务端。
  - **重点难点**：制定数据结构（JSON）。需要实现一套简化的算法来计算“水流蔓延到哪个格子”和“500个人群的坐标移动”，并负责把这些数据存入数据库以支持回放。
  - 例子见**附件A**
- **B：UE4 场景与表现组**
  - **核心任务**：搭建地铁站场景，实现接收数据并渲染。
  - **重点难点**：大量的人群仿真（tip from llm：不要用 Character 类做人（带不动），要研究 **VAT（顶点动画贴图）** 或者 **ISM** 技术来渲染大规模人群）；搞定逼真的水面和动态可控烟雾效果。
- **C：前端交互**
  - **核心任务**：写网页 UI。
  - **重点难点**：设计控制面板、时间轴进度条、键鼠操作逻辑手感等。负责把 UE 的视频流嵌入网页，并把鼠标键盘操作转发给 UE。

---

附件A 数据*结*构示例

**1. 初始化配置接口 (POST `/api/init`)**

- **用途**：仿真开始前，告诉 UE4 要演练什么场景。
- **数据包示例**：

```
{
  "scenario_id": "metro_fire_01",//编号
  "map_level": "Station_A_Platform", //选用哪张地图
  "total_people": 500,//人数
  "disaster_config": {
    "type": "fire_water_mix", // 灾害类型：复合灾害
    "fire_origin": { "x": 1200, "y": -500, "z": 100 }, // 起火点坐标
    //补充：火势大小、烟雾类型（毒性）
    "water_origin": { "id": "zone_entrance_02", "flow_rate": 50.0 } // 进水点
    //补充：总水量、进水速度
  }
}
```

**2. 实时帧数据接口 (GET `/api/frame_update`)**

- **用途**：这是最核心的接口。UE4 每秒会轮询多次（或通过 WebSocket 接收），每一帧包含**全场状态快照**。
- **数据包结构**：

```
{
  "timestamp": 15.5, // 仿真进行到了第 15.5 秒
  "status": "running", // 状态：running, paused, finished

  // --- 1. 环境数据（水与烟）---
```

```
  // 我们将地图划分为不同区域(Zone)或网格(Grid)，只传输有变化的区域
  "environment": [
    {
      "zone_id": "staircase_01",
      "water_level": 0.5,      // 水深 0.5米 (UE4据此调整水面高度)
      "smoke_density": 0.8,    // 烟雾浓度 80% (UE4调整粒子透明度)
      "smoke_type": "toxic"    // 毒性烟雾 (UE4显示为黑绿色)
    },
    {
      "zone_id": "platform_center",
      "water_level": 0.0,
      "smoke_density": 0.1,
      "smoke_type": "normal"
    }
  ],

  // --- 2. 人群数据 (最占带宽的部分) ---
  // 数组中每个对象代表一个乘客
  "agents": [
    {
      "id": 101,
      "pos": [1250, -400, 10], // [x, y, z] 坐标
      "dir": 90.0,             // 朝向角度
      "state": 2,              // 状态码: 0=站立, 1=奔跑, 2=跌倒, 3=半身入水, 4=完全
淹没
      "health": 80             // 健康值 (可选, 用于UI显示)
    },
    {
      "id": 102,
      "pos": [1300, -420, 10],
      "dir": 95.5,
      "state": 1,
      "health": 100
    }
    // ... 更多人
  ],

  // --- 3. 应对系统与指令 (大模型生成的决策) ---
  // UE4 需根据这些ID高亮显示对应的设备或NPC
  "events": [
    {
      "type": "staff_action",
      "target_id": "staff_01",  // 1号站务员
      "action": "guide_evacuation", // 动作: 指引疏散
      "msg": "请大家往A出口跑！"    // 气泡文字
    },
    {
      "type": "device_action",
      "target_id": "gate_05",   // 5号闸机
      "action": "force_open"    // 动作: 强制敞开
    }
  ],

  // --- 4. 实时统计 (给 Web UI 画图表用的, UE4可忽略) ---
```

```
    "statistics": {
      "total_evacuated": 50,       // 已逃生人数
      "in_water_deep": 12,         // 深水区人数
      "avg_exposure_time": 4.5     // 平均暴露在毒烟中的时间
    }
  }
```

**数据库相关**

通过根据编号或过程数据帧存储到数据库，我们可以存储多次仿真且随时可以切换到任一仿真的任意时间片，只需网页端用户发出请求，后端调取数据库交给ue渲染再发送给前端即可，无需后端仿真模块再次运行